Cloud Architecture Report — Agricultural Data Analysis

Project: `Agricultural_Data_Analysis` (ADS — Agricultural Dashboard & Prediction Service)
Report date: 2026-04-16
Status: Production (frontend on Vercel; backend on EC2 at `https://agri-intel.rvedire.com`)

This report documents the cloud architecture that has actually been adopted and deployed — not a whiteboard vision. Every service below has real data flowing through it today.

---

1. Executive summary

ADS is a single-developer project with a multi-tier footprint:

- **Three data planes** (raw parquet lake, relational store, model registry) backed by four AWS services (S3, RDS, EC2, Athena).
- **Two compute planes** (scheduled ETL + prediction serving) colocated on one `t3a.small` EC2 instance.
- **Two delivery planes** (static Next.js on Vercel edge + FastAPI on EC2) fronted by a single custom domain.
- **Serverless glue** (SNS for alerting, IAM for cross-service auth).
- **Cost target of ~$22/month** is held through strict separation of training (local PC) and serving (EC2).

The design is deliberately conservative — no Kubernetes, no managed ML platform, no Lambda sprawl. The architectural bet is that a cron-driven ETL + a long-running FastAPI process is sufficient at the current data volume (~870K county records, ~290K accuracy rows, 60 pickled models), and that preserving portability is worth more than autoscale.

---

2. The AWS services in use

Service	Purpose in this project	Instance / tier	Region
S3	Raw parquet lake (USDA	Bucket	`us-east-2`

Service	Purpose in this project	Instance / tier	Region
	NASS partitioned by state), Athena-optimized layout, model artifact registry, pickled model fallback	`usda-analysis-datasets`, Standard tier	
EC2	Hosts both the scheduled ETL pipeline (cron) *and* the FastAPI prediction service (systemd unit `ag-prediction`). Single instance.	`t3a.small` (2 vCPU, 2 GB RAM), Amazon Linux 2023	`us-east-2`
RDS (PostgreSQL)	Transactional store for forecasts, accuracy tables, futures/WASDE/DXY/ fertilizer/drought/RMA/CRP/ exports, feature snapshots	`ag-dashboard`, `db.t4g.micro` (2 vCPU, 1 GB RAM), Publicly Accessible = Yes (IP-whitelisted)	`us-east-2`
Athena	Ad-hoc SQL over S3 parquet lake (workgroup `usda-dashboard`, database `usda_agricultural`, table `quickstats_data`)	Serverless, scan-priced	`us-east-2`
SNS	Pipeline alerting — ETL/ ingest/inference success & failure notifications	Topic `usda-pipeline-alerts`	`us-east-2`
IAM	EC2 instance role for S3 read/write + SNS publish; scoped API keys for Vercel→EC2	—	global
Route 53 / external DNS	`agri-intel.rvedire.com` → EC2 Elastic IP, TLS via Let's Encrypt on the EC2 host	—	—

Everything runs in a single region (`us-east-2`) and a single AZ — no cross-AZ redundancy. This is a considered trade-off: the blast radius of an AZ failure is "forecasts are stale until the next cron," which is acceptable for a class-project/demo use case.

---

3. The three data planes

3.1 Raw data lake — S3

The USDA QuickStats API returns ~870K county records per full pull. These are transformed to parquet and uploaded to S3 in **two layouts**:

```
s3://usda-analysis-datasets/
├── survey_datasets/
│   ├── partitioned_states_counties/      # Browser-friendly (per-state files)
│   │   ├── state_01_alabama.parquet
│   │   ├── state_05_arkansas.parquet
│   │   └── ...
│   └── athena_optimized_counties/        # Hive-style partitions for Athena
│       └── state_fips=19/
│           └── year=2024/
│               └── data.parquet
└── models/
    ├── price/          # 18 PriceEnsemble pickles + .sig sidecars
    ├── acreage/        # 4 AcreageEnsemble pickles + .sig sidecars
    └── yield/          # 60 YieldModel pickles + .sig sidecars
```

Why dual layout? The Next.js frontend reads parquet directly from S3 via `hyparquet` (saves a round-trip through the API), which wants one file per state. Athena wants Hive-style partitioning. Keeping both means the frontend stays fast *and* analysts can SQL-query the same underlying data.

Upload integrity. `pipeline/upload_to_s3.py` uses `ChecksumAlgorithm=SHA256` + `ContentLength` verification on every upload. The `incremental_check.py` / `manifest.json` pattern ensures the `last_success` pipeline stamp only advances *after* a clean S3 upload, not before — so a failed upload can't permanently suppress a re-ingestion attempt. This was hardened in the 2026-04-16 audit.

3.2 Relational store — RDS PostgreSQL

The RDS instance holds anything that needs transactional semantics or is too small to warrant a parquet file. Schema grouped by module:

Module	Tables
Module 02: Price forecasting	`futures_daily`, `wasde_releases`, `price_forecasts`, `ers_production_costs`, `dxy_daily`

Module	Tables
Module 03: Acreage prediction	`acreage_forecasts`, `acreage_accuracy`, `ers_fertilizer_prices`, `drought_index`, `rma_insured_acres`, `crp_enrollment`, `export_commitments`
Module 04: Yield forecasting	`soil_features`, `feature_weekly`, `yield_forecasts`, `yield_accuracy`

Schema evolution is managed by **Alembic migrations** (`backend/alembic/versions/`) — seven migrations to date. The most recent three (`005_yield_accuracy`, `006_widen_commodity_col`, `007_nullable_model_forecast`) were deployed during the WI-1/WI-2/WI-3 accuracy-table pipeline work.

Sizing is deliberately tiny: `db.t4g.micro` handles ~290K yield accuracy rows and ~800 acreage accuracy rows without stress. The bottleneck, if any, would be connection count — the FastAPI process uses async SQLAlchemy with a bounded pool, and the ETL scripts use sync sessions that close after each run.

Security posture. The instance is `Publicly Accessible = Yes` for local-dev convenience (developer's home IP is whitelisted in the security group). This is acceptable because:

- No PII or secrets in this database — only public USDA data.
- Credentials are in `.env` and not committed.
- The app-level user `ag_app` has only DML privileges, not DDL (migrations run under a separate admin credential).

For a real production deployment this should be flipped to VPC-only with an SSM tunnel; that's flagged as deferred in CLAUDE.md.

3.3 Model registry — S3 + HMAC-signed pickles

Trained models are pickled with the dataclass pattern (`PriceEnsemble`, `AcreageEnsemble`, `YieldModel`) and uploaded to `s3://usda-analysis-datasets/models/{module}/{commodity}/[horizon_N|week_N]/`. Each `.pkl` has an adjacent `.sig` sidecar — HMAC-SHA256 over the artifact bytes, keyed by `MODEL_SIGNING_KEY`.

The FastAPI backend calls `ensure_verified_or_fail()` at load time. Under `MODEL_REQUIRE_SIGNED=1` the server refuses to unpickle unverified artifacts. This closes the pickle-deserialization attack surface — anyone with write access to the S3 prefix (CI, a rotated IAM key, a misconfigured bucket policy) would otherwise be able to achieve RCE on the prediction host.

Training → S3 → EC2 propagation:

```
Local PC (trains)                S3 model registry                EC2 (serves)
─────────────────                ─────────────────                ────────────
train.py / train_acreage.py      models/{module}/{c}/             _load_{m}_models()
train_yield.py        ──upload──▶  {path}/model.pkl  ─fallback▶  boto3.download
                                 {path}/model.pkl.sig               + HMAC verify
```

The model-load path tries local disk first, then falls back to S3. This means an EC2 restart re-hydrates the model cache from S3 — the instance is effectively stateless at the model layer.

---

4. The two compute planes

4.1 Scheduled ETL (EC2 cron)

Eight cron modes, all driven by `pipeline/cron_runner.sh`:

Mode	Cadence	Purpose
`(default / NASS)`	Monthly, 15th @ 6am CT	Full USDA QuickStats re-ingest
`--daily`	Weekdays @ 12pm CT	CME futures (Yahoo Finance) + FRED DXY
`--monthly-wasde`	12th of month @ 2pm CT	WASDE PSD pull + price inference + ag-prediction restart
`--annual-ers`	Jan 15 @ 10am CT	ERS production costs (Excel)
`--annual-acreage`	Feb 1 @ 12pm CT	Acreage inference + ag-prediction restart
`--quarterly-fertilizer`	Jan/Apr/Jul/Oct 1	ERS fertilizer prices (FRED)
`--weekly-yield`	Thursdays @ 10am CT	NOAA + NASA POWER + Drought + NASS conditions → yield inference
`--weekly-drought`	Thursdays @ 11am CT	USDM state-level DSCI refresh (offset to avoid table-write collision with yield)
`--annual-rma`	Mar 15 @ 10am CT	RMA insured acres

Mode	Cadence	Purpose
`--annual-crp`	Apr 1 @ 10am CT	FSA CRP enrollment/expirations
`--weekly-exports`	Thursdays @ 2pm CT	FAS export sales

Every mode acquires a per-mode `flock` on `/tmp` — different modes run concurrently but two runs of the same mode are serialized. This closes a concurrency bug where the 10am yield + drought jobs shared a DB table and could collide.

All ETL output flows to either S3 (parquet) or RDS (insert/upsert). Every script emits an SNS message (success or failure) to `usda-pipeline-alerts`; the developer subscribes to it by email.

4.2 Prediction API (EC2 systemd)

Service: `ag-prediction.service` — a systemd unit running `uvicorn backend.main:app` on port 8000. Auto-restart on failure. Reverse-proxied by Caddy or nginx on the same host, terminating TLS for `https://agri-intel.rvedire.com`.

Lifespan semantics. At startup, `lifespan()` in `backend/main.py` calls three loaders:

1. `_load_models()` — 18 price ensembles (3 commodities × 6 horizons)
2. `_load_acreage_models()` — 4 acreage ensembles (corn, soybean, wheat_winter, wheat_spring — plus a generic wheat alias the frontend still uses)
3. `_load_yield_models()` — 60 yield models (3 crops × 20 weeks)

Total: 82 models, ~50–150 MB in resident memory. Comfortable inside 2 GB. Each loader falls back to S3 if the local artifact is missing.

Async correctness. Endpoints that hit `psycopg2` (sync) wrap feature construction in `asyncio.to_thread` (`backend/routers/price.py`). Without this wrapper, every `GET /price/` would block the uvicorn event loop on a synchronous DB read — a latency cliff under any concurrency.

Endpoints exposed (v0.4.0):

```
GET  /health
GET  /api/v1/predict/price/                    ?commodity&horizon
GET  /api/v1/predict/price/probability         ?commodity&horizon&threshold
GET  /api/v1/predict/price/wasde-signal        ?commodity
GET  /api/v1/predict/price/history             ?commodity&horizon
GET  /api/v1/predict/acreage/                  ?commodity
GET  /api/v1/predict/acreage/states            ?commodity
GET  /api/v1/predict/acreage/accuracy          ?commodity
GET  /api/v1/predict/acreage/price-ratio
GET  /api/v1/predict/yield/                    ?crop&fips&week
GET  /api/v1/predict/yield/map                 ?crop&week
GET  /api/v1/predict/yield/history             ?crop&fips
GET  /api/v1/predict/yield/accuracy            ?crop
GET  /api/v1/predict/yield/metadata            ?crop           (gate status for UI)
GET  /api/v1/market/*                          (realized CME, DXY, etc.)
```

CORS is locked to the Vercel frontend origin by `settings.CORS_ORIGINS`. `GET` is the only allowed method — writes come from the ETL layer only.

---

5. The two delivery planes

5.1 Frontend — Vercel

The Next.js 16 app (`web_app/`) is deployed to Vercel. Build pipeline:

```
GitHub push to main
   └─ Vercel detects commit
      └─ npm ci && npm run build (Turbopack)
         └─ Edge-deploy static + serverless functions
            └─ Proxies /api/data → S3 parquet     (hyparquet browser-side)
                       /api/athena → Athena boto3
                       /api/predict/* → EC2 FastAPI
```

Environment variables set in Vercel:

- `NEXT_PUBLIC_PREDICTION_API_URL=https://agri-intel.rvedire.com`
- AWS creds (read-only S3 + Athena) for the `/api/athena` serverless route

Three fallback tiers for the dashboard's raw-data read path:

1. **Primary:** browser reads parquet directly from S3 (`hyparquet`, no round-trip).

2. **Fallback:** `/api/data` proxies through Next.js (for CORS-locked browsers).
3. **Advanced:** `/api/athena` for SQL queries that can't be expressed as parquet filters.

5.2 Backend — EC2 FastAPI behind reverse proxy

`https://agri-intel.rvedire.com` → EC2 Elastic IP → Caddy → uvicorn:8000. TLS via Let's Encrypt auto-renewal. CORS allowlist includes the Vercel preview URLs plus the production origin.

---

6. Cross-cutting concerns

6.1 Observability

- **Pipeline events:** SNS topic `usda-pipeline-alerts` — every cron mode publishes both failure *and* success messages. Developer subscribes by email.
- **Application logs:** `backend/main.py` and each ETL script use the `setup_logging()` helper from `backend/etl/common.py`, writing to stderr with a consistent format. systemd captures these to journald; cron writes to `pipeline/logs/cron_YYYYMMDD_HHMMSS.log`.
- **Metrics:** per-module `metrics.json` artifacts written alongside each pickle (train RRMSE, val RRMSE, test RRMSE, baselines, feature importances, gate pass/fail). These become the single source of truth for the `/yield/metadata` endpoint that powers the frontend's "Experimental" vs "Production" badge.

6.2 Security

- **Secrets:** `.env` at project root, not committed. Loaded via `pydantic-settings`. EC2 reads from `~/Agricultural_Data_Analysis/backend/.env`.
- **IAM:** EC2 instance role scoped to `s3:GetObject`, `s3:PutObject` on `usda-analysis-datasets` and `sns:Publish` on the pipeline-alerts topic. No `*` wildcards.
- **Pickle signing:** HMAC-SHA256 sidecars on all model artifacts (§3.3). `MODEL_REQUIRE_SIGNED=1` is the production posture.
- **SQL:** SQLAlchemy ORM + parameterized queries everywhere. Two places accept a column name as input (`_query_ers_cost`, `_query_fertilizer_price`) — both gated by a hard-coded `frozenset` allowlist that raises on unknown values.
- **Deferred:** `.env` rotation flagged but deferred — the committed placeholder-style values aren't live credentials.

6.3 Training/serving separation

A key design constraint is **no ML training on EC2**. Training happens on the developer's local PC (or, with the notebooks described in deliverable 3, on Colab), which has the CPU/RAM headroom for walk-forward training across 60 models. The training script uploads artifacts to S3; the EC2 serving process downloads them on restart. This keeps EC2 costs at ~$6/mo — an `m5.large` sized for training would triple the bill.

6.4 Data contracts between tiers

The hardest bugs in this project have come from contract drift between the ML layer and the frontend. Current contracts:

- **Acreage response shape:** renamed `forecast_acres_millions` → `forecast_acres` on 2026-04-16 because the stored value was always raw acres — the `_millions` suffix was misleading the UI into double-scaling.
- **Wheat commodity key:** frontend sends `wheat`; backend aliases it to the generic wheat artifact even though training produces `wheat_winter` and `wheat_spring` separately. The alias layer is inside `_load_acreage_models()`.
- **Yield gate status:** the `/yield/metadata` endpoint exposes `gate_status` as `pass` / `partial` / `fail` so the UI can render an "Experimental" badge rather than hiding borderline models.

These contracts are tested by frontend integration (manual, for now) rather than by a type bridge. Flagged for future work.

---

7. Cost profile

Monthly bill, April 2026:

Line item	Cost	Notes
RDS `db.t4g.micro`	~$15	20 GB gp3 storage included
EC2 `t3a.small` (serving + cron only)	~$6	30 GB gp3 root
S3 storage (~10 GB parquet + ~500 MB models)	<$1	Standard tier
S3 requests	<$0.50	Model loads on restart are the dominant source

Line item	Cost	Notes
Athena scans	<$0.50	Small workgroup, mostly filtered queries
SNS publishes	negligible	
Data transfer out	<$1	Vercel edge pulls parquet
Total	**~$22/mo**	

Vercel and the custom domain are separate (free tier for Vercel hobby + ~$1/mo for `rvedire.com` via namecheap).

---

8. What we explicitly don't have

Calling out absences so readers don't infer them:

- **No VPC design.** Everything lives in the default VPC. For a real production deployment we'd put RDS in private subnets and proxy EC2 through an ALB — not needed at current scale.
- **No container orchestration.** No Docker, no ECS, no Kubernetes. `ag-prediction.service` is a systemd unit running `uvicorn` directly.
- **No Lambda.** Every scheduled job runs on the EC2 host. Lambda was considered for ETL but rejected — `yfinance` + `pandas` + `pyarrow` together bust the 250 MB Lambda deployment limit, and cold-start latency for the bulk-ingest jobs would not help.
- **No managed MLflow / SageMaker / Vertex AI.** Training is Python scripts + pickle files. The notebooks in deliverable 3 introduce local MLflow to experimentation but do not require a hosted tracking server for the serving path.
- **No CDN in front of the API.** The FastAPI responses are dynamic and per-user-query — caching would need per-endpoint rules that aren't worth building yet.
- **No CI/CD for the backend.** Local PC → `scp` + `systemctl restart` → done. The frontend has Vercel's built-in CI.
- **No feature store.** `feature_weekly` in RDS is the closest analogue, but it's a persistence table, not a feature-serving layer. Online inference rebuilds features from raw tables on every call.

These are conscious omissions. The project optimizes for two metrics — monthly cost and developer cognitive load — and adding any of the above would compromise both without improving current user-visible behavior.

---

9. Deployment topology (quick reference)

```
┌──────────────────────────────┐          ┌──────────────────────────────┐
│  Developer PC (local)        │          │  End user (browser)          │
│  - Training (Python CLI)     │          │                              │
│  - Model artifacts produced  │          └──────────────────────────────┘
└──────────────────────────────┘                     │
           │                                         │
           │ aws s3 sync                             │ HTTPS
           ▼                                         ▼

┌──────────────────────────────┐          ┌──────────────────────────────┐
│  S3 us-east-2                │          │  Vercel (Edge)               │
│  usda-analysis-datasets      │◀─────────┤  Next.js 16 SSR/SSG          │
│  - survey_datasets/          │ parquet  │  /api/data, /api/athena      │
│  - models/ (.pkl + .sig)     │          │  /api/predict proxy          │
└──────────────────────────────┘          └──────────────────────────────┘
           │                                         │
           │                                         │
           │                                         │ HTTPS (agri-intel)
           │  boto3 fallback                         │
           ▼                                         ▼

┌──────────────────────────────────────────────────────────────────────┐
│  EC2 t3a.small (us-east-2a)                                          │
│  ┌──────────────────────────────┐   ┌──────────────────────────────┐  │
│  │ systemd: ag-prediction.svc   │   │ cron: pipeline/cron_runner.sh│  │
│  │ uvicorn :8000 (FastAPI)      │   │ 9 scheduled modes            │  │
│  │ 82 models in memory          │   │ ETL scripts (backend/etl/*)  │  │
│  └──────────────────────────────┘   └──────────────────────────────┘  │
│                 │ asyncpg / psycopg2                │ psycopg2         │
└─────────────────┴───────────────────────────────────┴──────────────────┘
                  │                                   │
                  ▼                                   ▼
        ┌──────────────────────────────────────────────────┐
        │  RDS PostgreSQL (db.t4g.micro, us-east-2)        │
        │  ag-dashboard / ag_dashboard / ag_app            │
        │  15 tables across 3 modules                      │
        └──────────────────────────────────────────────────┘
                          │
                          │ SQL
                          ▼
                 ┌────────────────┐          ┌──────────────────┐
                 │  Athena        │          │  SNS topic       │
                 │  usda_agri...  │          │  pipeline-alerts │
                 │  (S3-backed)   │          │  → email         │
                 └────────────────┘          └──────────────────┘
```

(See `cloud-architecture-diagram.html` for the interactive diagram.)

---

10. Open items / roadmap

1. **Rotate `.env` secrets** before any public demo (deferred in 2026-04-16 audit).
2. **VPC migration** of RDS — flip public accessibility off and route through SSM/bastion.
3. **Calibration pass on yield intervals** — under-covering 80% PIs (corn 0.636 test, soy 0.541 test, wheat 0.427 test). Likely requires a larger val set or per-crop conformal split.
4. **Wheat yield model re-work** — currently fails the 10% gate across all weeks (hidden from UI via metadata endpoint). Candidate avenues: split winter/spring yield like we did for acreage, or add class-specific weather windows.
5. **CI/CD for backend** — `gh actions` → `scp` + `systemctl restart` would cost zero and remove the last manual deploy step.
6. **Notebook-driven experimentation** (deliverable 3 of this session) — unlocks community model submissions. If a submitted model beats baselines, it can be dropped into the existing model registry without schema change.

End of report.